Лабораторна робота №3

Тема: Анотації

Як розробник, що розширює цю кодову базу, я хочу мати невеликий набір кастомних анотацій, щоб зрозуміти весь спектр того, що власні нотації можуть робити у Spring Boot застосунку.

Опис

Реалізувати **три** кастомні анотації у своєму проєкті предметної області, кожна з яких демонструє свій унікальний механізм.

Тільки одна повинна бути Bean Validation constraint-анотацією (за патерном @ValidSkuCategory, який було розглянуто на лекції). Інші три мають бути трьох інших типів, перелічених нижче. Повторне використання одного й того самого механізму двічі не дозволяється.

Тип 1 — Bean Validation Constraint

Спроєктувати власну constraint-анотацію, застосовану до DTO у предметній області. Це має бути дійсно нове правило — не скопійований @ValidSkuCategory зі зміненими назвами полів.

Acceptance criteria 1:

GIVEN анотацію на рівні класу з @Constraint(validatedBy = ...)

WHEN DTO з цією анотацією проходить валідацію

THEN Bean Validation автоматично знаходить і викликає потрібний клас валідатора, без жодного ручного створення екземпляра де-небудь у коді застосунку.

Acceptance criteria 2

GIVEN DTO, поля якого задовольняють кастомне бізнес-правило

WHEN він валідується (напряму через Validator, або через реальний запит)

THEN жодного порушення для цього constraint-а не виникає.

Acceptance criteria 3:

GIVEN DTO, поля якого порушують кастомне бізнес-правило

WHEN він валідується

THEN виникає порушення (violation)

AND його повідомлення чітко пояснює, що саме не так (а не загальне повідомлення на кшталт "invalid").

Тип 2 — Композитна анотація (Composed / Meta-Annotation)

Об'єднати дві чи більше вже існуючі анотації, які регулярно використовуються разом у коді, в одну повторно використовувану кастомну анотацію — той самий патерн, який Spring використовує для @SpringBootApplication чи @RestController.

Acceptance criteria 4:

GIVEN щонайменше дві анотації, які регулярно використовуються разом у коді (наприклад, mapping-анотація + response status, або валідаційна анотація + документаційна анотація)

WHEN створється нова анотація, яка сама позначена обома цими анотаціями

THEN застосування вашої нової анотації до методу/класу дає точно таку саму поведінку під час виконання, як і застосування оригінальних анотацій окремо.

Acceptance criteria 5:

GIVEN ендпоінт або клас, що використовує композитну анотацію

WHEN він перевіряється (наприклад, через @WebMvcTest або прямий виклик)

THEN тест підтверджує ТАКУ САМУ поведінку, яку дали б оригінальні, нескомбіновані анотації (наприклад, коректний HTTP статус, коректний routing)

Тип 3 — Кастомна анотація-резолвер аргументів MVC

Створити анотацію, яку резолвить HandlerMethodArgumentResolver, щоб метод контролера міг отримувати значення, похідне від контексту запиту (наприклад, ім'я поточного автентифікованого користувача, tenant ID із заголовка, розпарсене кастомне значення) без повторення логіки видобування в кожному методі.

Acceptance criteria 6:

GIVEN реалізацію HandlerMethodArgumentResolver

WHEN викликається supportsParameter(...) з параметром, що має вашу кастомну анотацію

THEN повертається true

І для параметра БЕЗ цієї анотації повертається false

Acceptance criteria 7:

GIVEN запит із контекстом, від якого залежить ваша анотація (наприклад, автентифікований користувач, заголовок, path variable)

WHEN викликається resolveArgument(...)

THEN повертається коректно видобуте значення правильного типу

Acceptance criteria 8:

GIVEN метод контролера з параметром, позначеним кастомною анотацією

WHEN надсилається реальний запит на цей ендпоінт

THEN резолвлене значення дійсно використовується всередині тіла методу

(а не просто резолвиться і відкидається)